Отчёт по лабораторной работе №3

Web-программирование: «Реализация простого сайта на Django»

Выполнил: Клименко Богдан Владимирович, ИСУ №408790, группа K3340
Среда: VS Code + Django

Цель работы:

Овладеть практическими навыками реализации backend-части веб-приложений с использованием Django REST Framework, включая создание моделей, сериализаторов, view-классов, а также документирование API и реализацию аутентификации.

Выбор варианта и проектирование моделей

В соответствии с вариантом №19 была разработана модель базы данных для учёта животных в зоопарке. Основные сущности:

- Животное
- Отдел
- Здание
- Вольер
- Сотрудник (смотритель, ветеринар)
- Рацион, тип рациона
- Зоопарк (для аренды)
- Место зимовки (для птиц)
- История кормления, назначения смотрителей/ветеринаров, история пересадок

Листинг моделей (main/models.py) представлен в приложении (см. Приложение А).

Реализация API средствами Django REST Framework

Сериализаторы

Для каждой модели создан сериализатор, обеспечивающий преобразование объектов в JSON и обратно. Пример для животного:

```
        class AnimalSerializer(serializers.ModelSerializer):
    habitat_name = serializers.CharField(source='habitat.name', read_only=True)
    department_name = serializers.CharField(source='department.name',
read_only=True)
    enclosure_number = serializers.CharField(source='current_enclosure.number',
read_only=True)

    class Meta:
        model = Animal
        fields = '__all__'
```

ViewSet'ы

Для всех моделей использованы ModelViewSet, что позволило автоматически получить стандартные CRUD-операции. Для реализации специфических запросов добавлены @action методы.

Пример для отдела:

```
class DepartmentViewSet(viewsets.ModelViewSet):
    queryset = Department.objects.all()
    serializer_class = DepartmentSerializer
    permission_classes = [permissions.IsAuthenticated]
    @action(detail=False, methods=['get'], url_path='animal-count')
    def animal_count(self, request):
        departments = Department.objects.annotate(animal_count=Count('animals'))
        data = [{'id': d.id, 'name': d.name, 'animal_count': d.animal_count} for
d in departments]
        return Response(data)
```

Маршруты

Все маршруты зарегистрированы через DefaultRouter в main/urls.py:

```
router = DefaultRouter()
router.register(r'habitats', HabitatViewSet)
```

```
router.register(r'departments', DepartmentViewSet)
router.register(r'buildings', BuildingViewSet)
router.register(r'enclosures', EnclosureViewSet)
router.register(r'staff', StaffViewSet)
router.register(r'animals', AnimalViewSet)
router.register(r'feeding-ration-types', FeedingRationTypeViewSet)
router.register(r'feeding-rations', FeedingRationViewSet)
router.register(r'zoos', ZooViewSet)
router.register(r'wintering-places', WinteringPlaceViewSet)
router.register(r'feeding-histories', FeedingHistoryViewSet)
router.register(r'caretaker-assignments', CaretakerAssignmentViewSet)
router.register(r'veterinarian-assignments', VeterinarianAssignmentViewSet)
router.register(r'transfer-histories', TransferHistoryViewSet)

urlpatterns = [
    path('', include(router.urls)),
]
```

Главный файл config/urls.py подключает маршруты приложения, а также эндпоинты аутентификации и документации.

Кастомные эндпоинты

Были реализованы следующие запросы:

1. **Количество животных в каждом отделе**
   GET /api/v1/departments/animal-count/
   Результат:

[{"id":5,"name":"Хищники","animal_count":3}, ...]

1. **Список животных в коммунальных вольерах**
   GET /api/v1/animals/communal/
   Возвращает животных, находящихся в вольерах с признаком is_communal=True.

2. **Список животных в том же здании, что и заданное животное**
   GET /api/v1/animals/{id}/same-building-animals/
   Пример для льва (ID=4) вернул тигра и леопарда.

3. **Список пустых вольеров**
   GET /api/v1/enclosures/empty/
   После заполнения базы вольеры оказались заняты, результат – пустой список.
4. **Отчёт по аренде животных**
   GET /api/v1/zoos/lease-report/
   Возвращает зоопарки-арендодатели с количеством арендованных животных и общей стоимостью.

   Все эндпоинты протестированы и работают корректно (скриншоты прилагаются в разделе 7).

Аутентификация и документация

Djoser и JWT

Для аутентификации использован пакет djoser в связке с djangorestframework-simplejwt. В settings.py настроены:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ]
}
```

Доступны эндпоинты регистрации, получения токена и управления пользователями.

Документация drf-yasg

Подключена интерактивная документация Swagger и ReDoc.

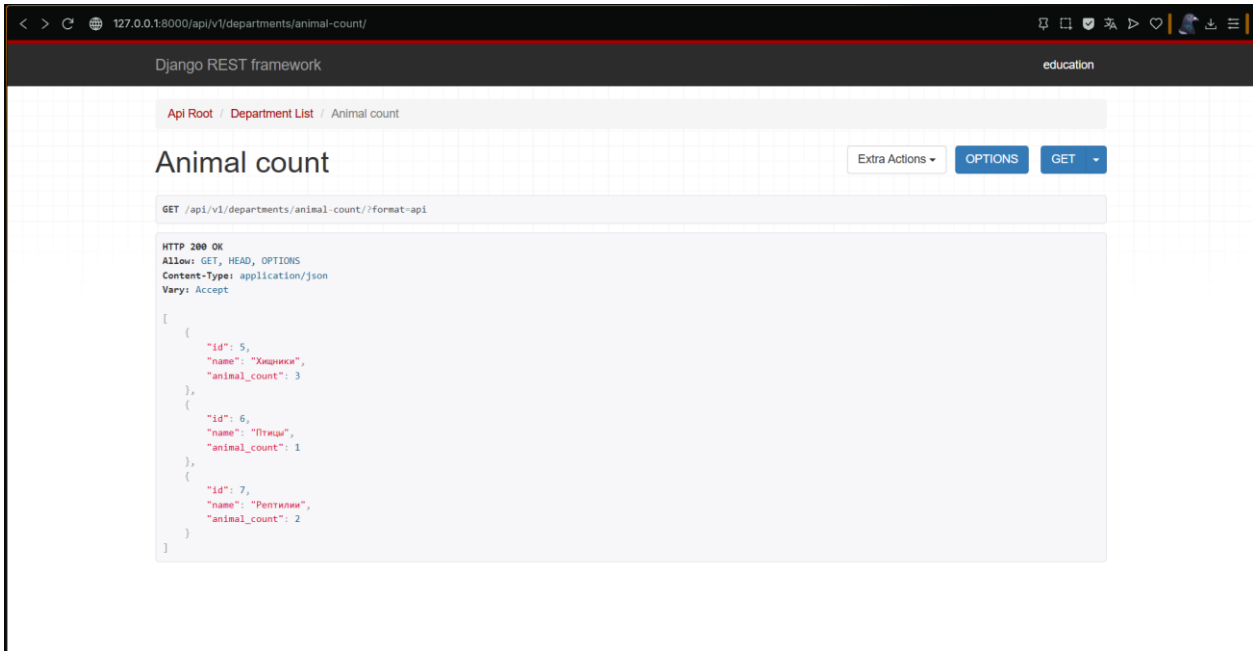
- Swagger UI: http://127.0.0.1:8000/swagger/
- ReDoc: http://127.0.0.1:8000/redoc/

Документация автоматически отображает все зарегистрированные эндпоинты и позволяет выполнять тестовые запросы.

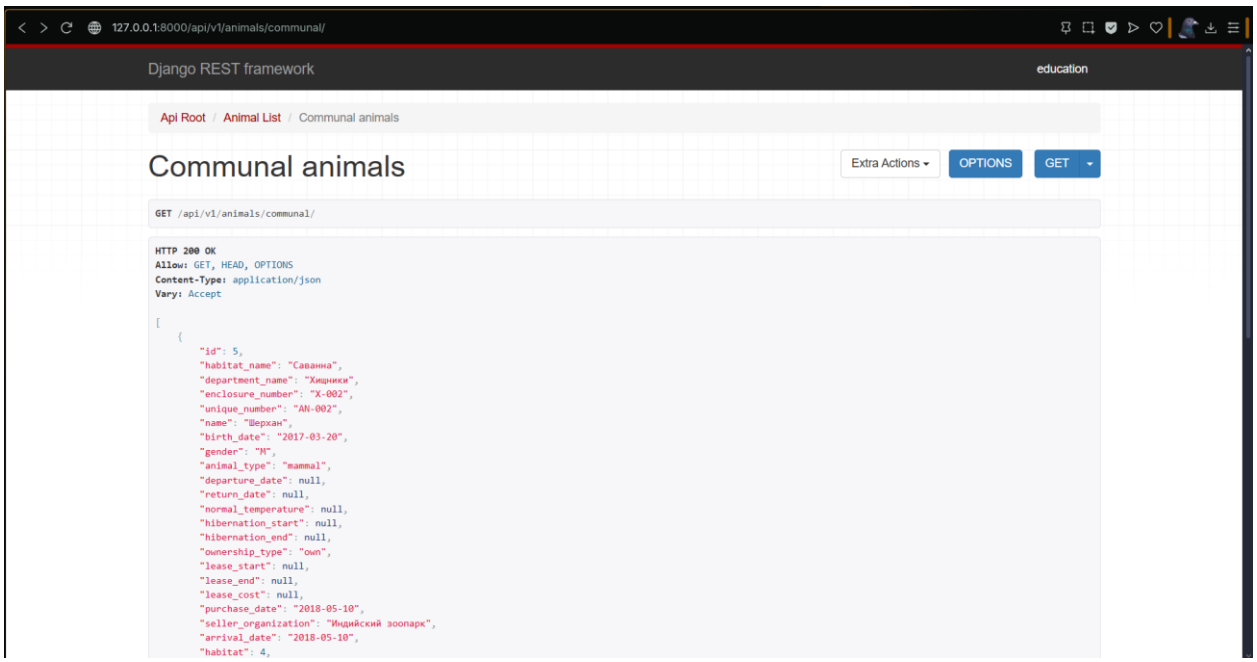
Тестирование работы API

Для проверки использовался браузер (при временном отключении аутентификации). Скриншоты результатов по каждому эндпоинту представлены ниже.

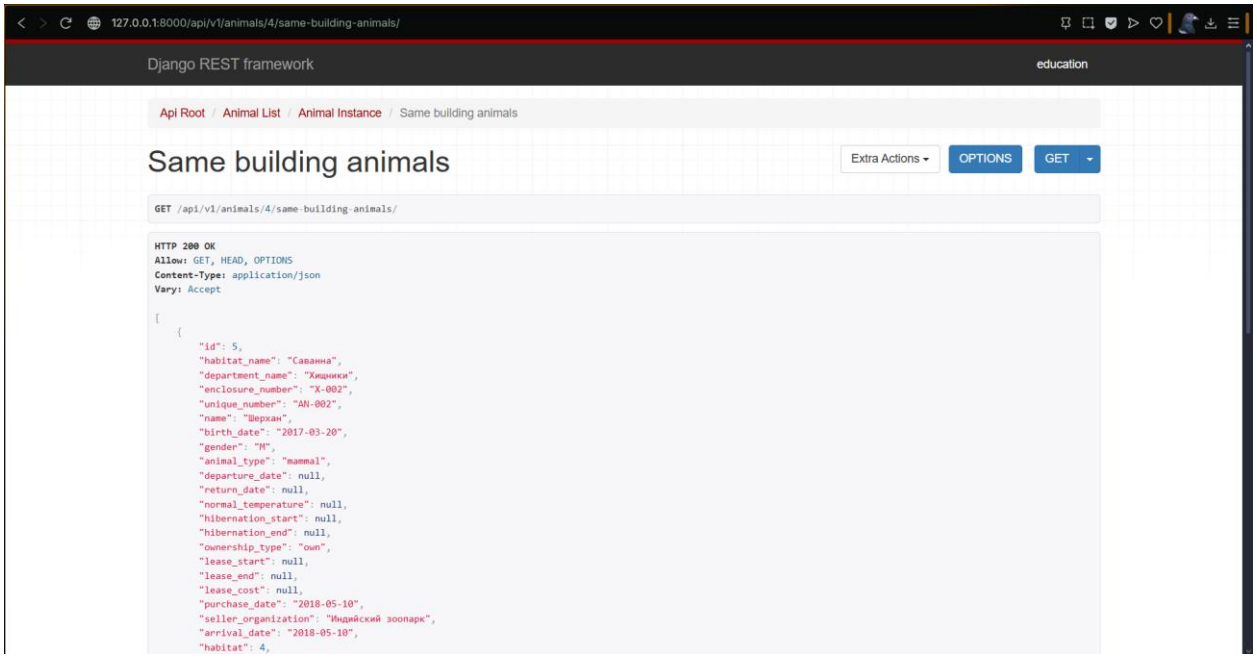
6.1. Количество животных в отделах



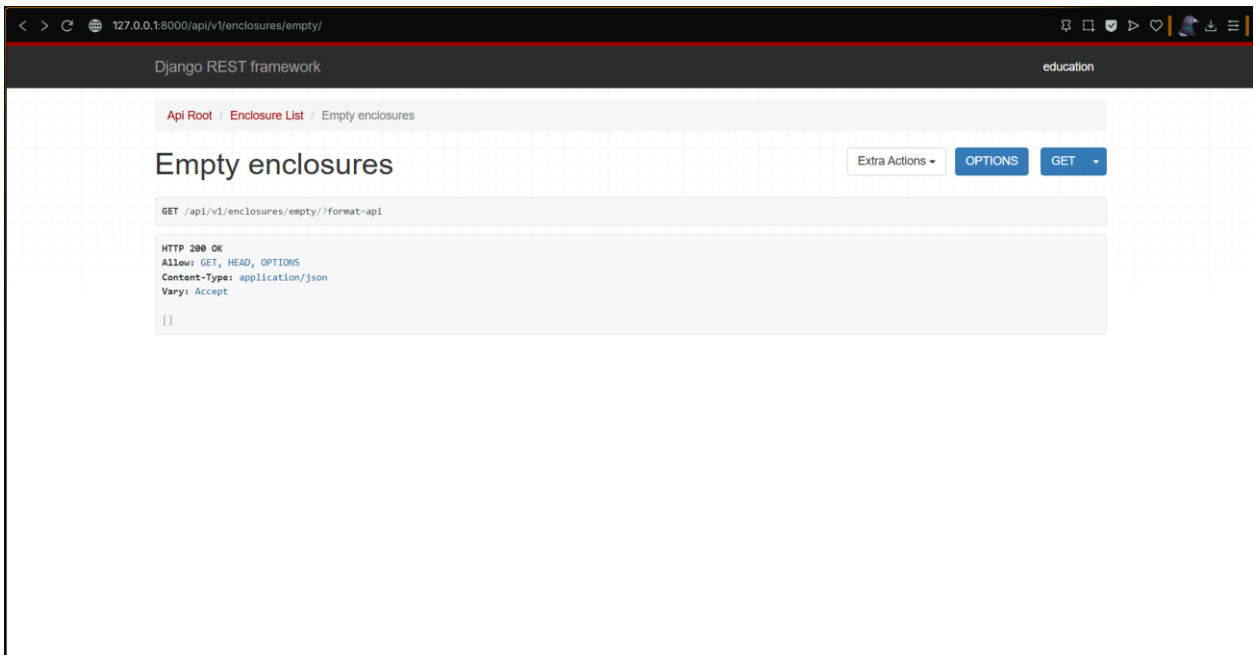
6.2. Животные в коммунальных вольерах



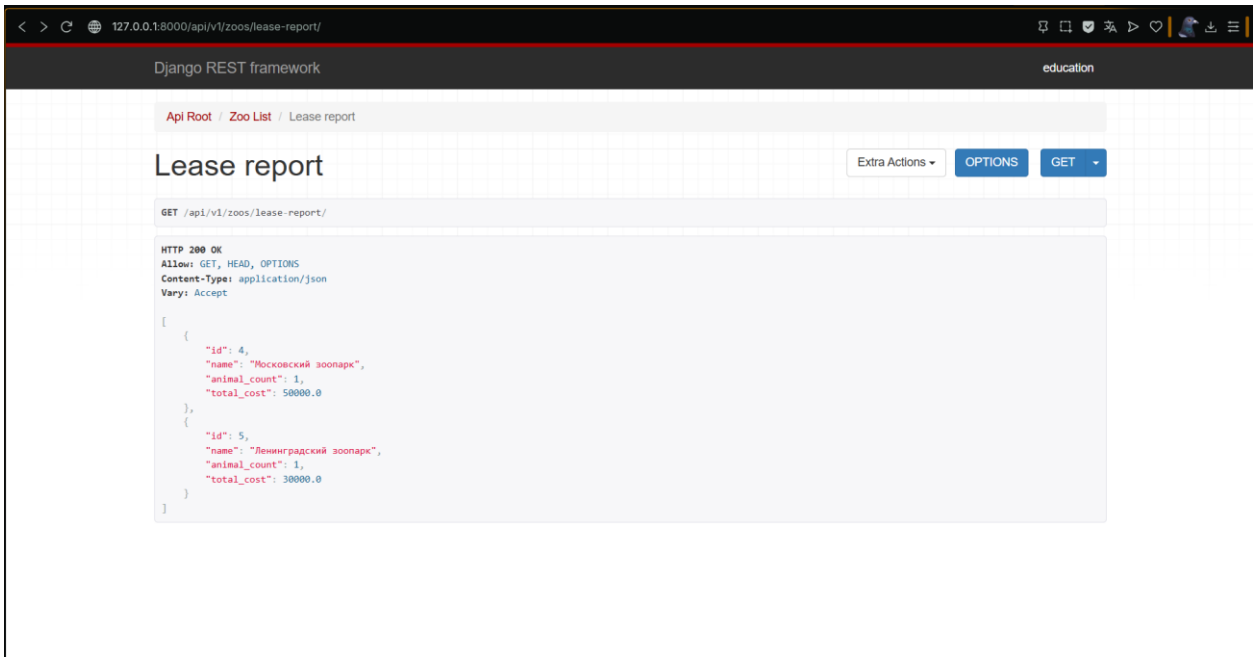
6.3. Животные в том же здании (лев ID=4)



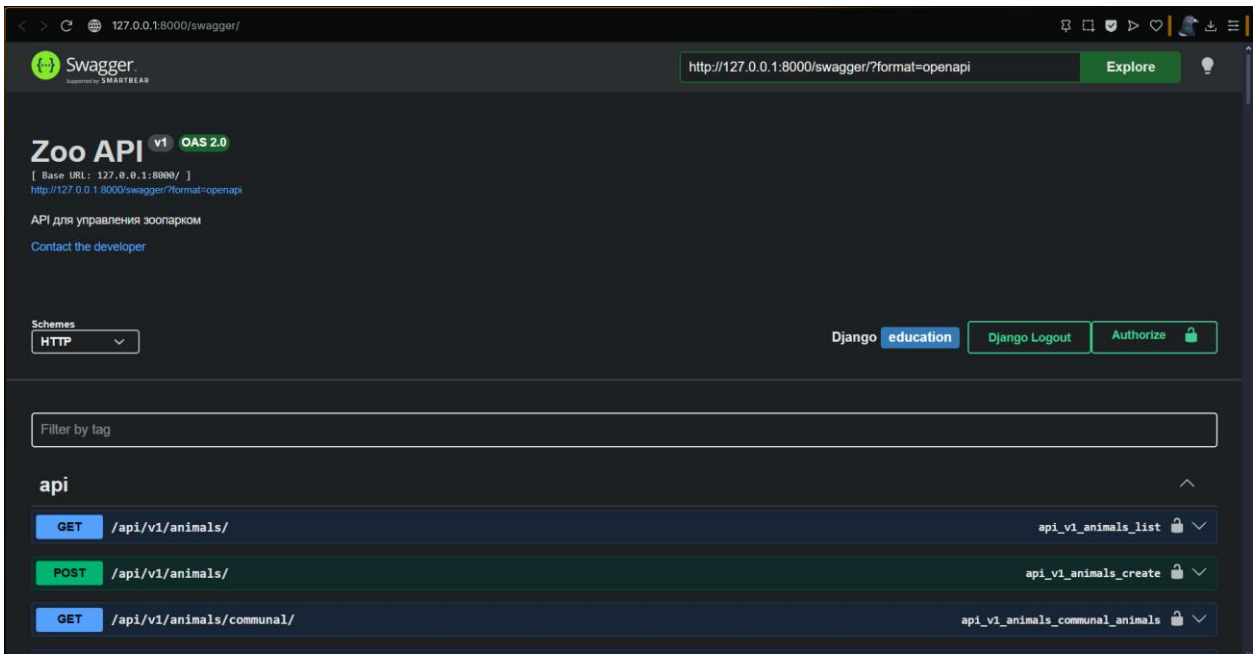
6.4. Пустые вольеры



6.5. Отчёт по аренде



6.6. Документация Swagger



Инструкция по развёртыванию

1. Клонировать репозиторий и перейти в папку simple_drf_project.
2. Создать виртуальное окружение:

   python -m venv venv

   venv\Scripts\activate # Windows
3. Установить зависимости:

   pip install -r requirements.txt
4. Применить миграции:

   python manage.py migrate
5. Загрузить тестовые данные (опционально):

   python fill_data.py
6. Создать суперпользователя:

   python manage.py createsuperuser
7. Запустить сервер:

   python manage.py runserver

8. Перейти по адресу http://127.0.0.1:8000/api/v1/ для доступа к API.

Вывод

В ходе работы была разработана серверная часть для системы учёта животных в зоопарке. Реализованы модели данных, сериализаторы, view-классы с кастомными методами, аутентификация на основе JWT и документация API. Все требуемые запросы из варианта работают корректно. Полученные навыки позволяют создавать полноценные REST API на Django и готовы к интеграции с фронтенд-приложением

Приложение А. Листинг файла models.py

```
from django.db import models
from django.contrib.auth.models import User  # для связи с пользователями, если
потребуется

# Справочник типов рациона
class FeedingRationType(models.Model):
    name = models.CharField(max_length=100, verbose_name='Название типа')

    class Meta:
        verbose_name = 'Тип рациона'
        verbose_name_plural = 'Типы рационов'

    def __str__(self):
        return self.name

# Рацион кормления
class FeedingRation(models.Model):
    number = models.CharField(max_length=20, unique=True, verbose_name='Номер
рациона')
    name = models.CharField(max_length=200, verbose_name='Название')
    ration_type = models.ForeignKey(FeedingRationType, on_delete=models.PROTECT,
verbose_name='Тип рациона')

    class Meta:
        verbose_name = 'Рацион'
        verbose_name_plural = 'Рационы'

    def __str__(self):
        return f"{self.number} - {self.name}"

# Зона обитания
class Habitat(models.Model):
    name = models.CharField(max_length=200, verbose_name='Название зоны')
    continent = models.CharField(max_length=100, verbose_name='Материк')
    country = models.CharField(max_length=100, verbose_name='Страна')
    description = models.TextField(blank=True, verbose_name='Характеристика')

    class Meta:
        verbose_name = 'Зона обитания'
        verbose_name_plural = 'Зоны обитания'

    def __str__(self):
        return self.name

# Зоопарк (для аренды/собственности)
class Zoo(models.Model):
    name = models.CharField(max_length=200, verbose_name='Название зоопарка')
    address = models.TextField(verbose_name='Адрес')
```

```
    contact_phone = models.CharField(max_length=20, blank=True,
verbose_name='Телефон')
    contact_email = models.EmailField(blank=True, verbose_name='Email')

    class Meta:
        verbose_name = 'Зоопарк'
        verbose_name_plural = 'Зоопарки'

    def __str__(self):
        return self.name

# Место зимовки для птиц
class WinteringPlace(models.Model):
    code = models.CharField(max_length=20, unique=True, verbose_name='Код')
    country = models.CharField(max_length=100, verbose_name='Страна')
    name = models.CharField(max_length=200, blank=True, verbose_name='Название
места')

    class Meta:
        verbose_name = 'Место зимовки'
        verbose_name_plural = 'Места зимовки'

    def __str__(self):
        return f"{self.code} - {self.country}"

# Отдел зоопарка
class Department(models.Model):
    name = models.CharField(max_length=100, verbose_name='Название отдела')
    description = models.TextField(blank=True, verbose_name='Описание')

    class Meta:
        verbose_name = 'Отдел'
        verbose_name_plural = 'Отделы'

    def __str__(self):
        return self.name

# Здание
class Building(models.Model):
    BUILDING_TYPE_CHOICES = [
        ('summer', 'Летнее'),
        ('winter', 'Зимнее'),
    ]
    name = models.CharField(max_length=100, verbose_name='Название здания')
    building_type = models.CharField(max_length=10,
choices=BUILDING_TYPE_CHOICES, verbose_name='Тип здания')
    department = models.ForeignKey(Department, on_delete=models.CASCADE,
related_name='buildings', verbose_name='Отдел')

    class Meta:
        verbose_name = 'Здание'
```

```
        verbose_name_plural = 'Здания'

    def __str__(self):
        return f"{self.name} ({self.get_building_type_display()})"

# Вольер
class Enclosure(models.Model):
    number = models.CharField(max_length=20, verbose_name='Номер вольера')
    department = models.ForeignKey(Department, on_delete=models.CASCADE,
related_name='enclosures', verbose_name='Отдел')
    building = models.ForeignKey(Building, on_delete=models.SET_NULL, null=True,
blank=True, related_name='enclosures', verbose_name='Здание')
    is_communal = models.BooleanField(default=False, verbose_name='Коммунальная
квартира')
    has_pool = models.BooleanField(default=False, verbose_name='Наличие
бассейна')
    has_equipment = models.BooleanField(default=False,
verbose_name='Дополнительное оборудование')
    has_indoor = models.BooleanField(default=False, verbose_name='Внутреннее
помещение')
    capacity = models.PositiveIntegerField(verbose_name='Вместимость')

    class Meta:
        verbose_name = 'Вольер'
        verbose_name_plural = 'Вольеры'
        unique_together = ('department', 'number')  # в пределах отдела номер
уникален

    def __str__(self):
        return f"Вольер {self.number} ({self.department.name})"

# Сотрудник (общая информация)
class Staff(models.Model):
    STAFF_TYPE_CHOICES = [
        ('caretaker', 'Смотритель'),
        ('vet', 'Ветеринар'),
    ]
    personnel_number = models.CharField(max_length=20, unique=True,
verbose_name='Табельный номер')
    full_name = models.CharField(max_length=200, verbose_name='ФИО')
    birth_date = models.DateField(verbose_name='Дата рождения')
    phone = models.CharField(max_length=20, verbose_name='Телефон')
    email = models.EmailField(blank=True, verbose_name='Email')
    staff_type = models.CharField(max_length=10, choices=STAFF_TYPE_CHOICES,
verbose_name='Тип сотрудника')

    class Meta:
        verbose_name = 'Сотрудник'
        verbose_name_plural = 'Сотрудники'

    def __str__(self):
```

```
        return f"{self.full_name} ({self.get_staff_type_display()})"

# Животное (основная модель)
class Animal(models.Model):
    GENDER_CHOICES = [
        ('M', 'Самец'),
        ('F', 'Самка'),
    ]
    ANIMAL_TYPE_CHOICES = [
        ('mammal', 'Млекопитающее'),
        ('bird', 'Птица'),
        ('reptile', 'Рептилия'),
    ]
    OWNERSHIP_CHOICES = [
        ('own', 'Собственность зоопарка'),
        ('lease', 'Аренда'),
    ]

    # Основные поля
    unique_number = models.CharField(max_length=50, unique=True,
verbose_name='Уникальный номер')
    name = models.CharField(max_length=200, verbose_name='Имя')
    birth_date = models.DateField(verbose_name='Дата рождения')
    gender = models.CharField(max_length=1, choices=GENDER_CHOICES,
verbose_name='Пол')
    animal_type = models.CharField(max_length=10, choices=ANIMAL_TYPE_CHOICES,
verbose_name='Тип животного')
    habitat = models.ForeignKey(Habitat, on_delete=models.PROTECT,
related_name='animals', verbose_name='Зона обитания')
    department = models.ForeignKey(Department, on_delete=models.PROTECT,
related_name='animals', verbose_name='Отдел')
    current_enclosure = models.ForeignKey(Enclosure, on_delete=models.PROTECT,
related_name='animals', verbose_name='Текущий вольер')

    # Поля для птиц
    wintering_place = models.ForeignKey(WinteringPlace,
on_delete=models.SET_NULL, null=True, blank=True, verbose_name='Место зимовки')
    departure_date = models.DateField(null=True, blank=True, verbose_name='Дата
улёта')
    return_date = models.DateField(null=True, blank=True, verbose_name='Дата
прилёта')   # было arrival_date


    # Дополнительные поля для рептилий
    normal_temperature = models.DecimalField(max_digits=5, decimal_places=2,
null=True, blank=True, verbose_name='Нормальная температура')
    hibernation_start = models.DateField(null=True, blank=True,
verbose_name='Начало спячки')
    hibernation_end = models.DateField(null=True, blank=True, verbose_name='Конец
спячки')
```

```
    # Информация о собственности
    ownership_type = models.CharField(max_length=5, choices=OWNERSHIP_CHOICES,
verbose_name='Тип собственности')
    owner_zoo = models.ForeignKey(Zoo, on_delete=models.SET_NULL, null=True,
blank=True, related_name='leased_animals', verbose_name='Зоопарк-владелец
(аренда)')
    lease_start = models.DateField(null=True, blank=True, verbose_name='Начало
аренды')
    lease_end = models.DateField(null=True, blank=True, verbose_name='Окончание
аренды')
    lease_cost = models.DecimalField(max_digits=10, decimal_places=2, null=True,
blank=True, verbose_name='Стоимость аренды')
    purchase_date = models.DateField(null=True, blank=True, verbose_name='Дата
покупки')
    seller_organization = models.CharField(max_length=200, blank=True,
verbose_name='Организация-продавец')
    arrival_date = models.DateField(verbose_name='Дата поступления в зоопарк')

    # Связи с сотрудниками (многие ко многим через промежуточные модели)
    caretakers = models.ManyToManyField(Staff, through='CaretakerAssignment',
related_name='animals_cared', limit_choices_to={'staff_type': 'caretaker'})
    veterinarians = models.ManyToManyField(Staff,
through='VeterinarianAssignment', related_name='animals_vetted',
limit_choices_to={'staff_type': 'vet'})

    class Meta:
        verbose_name = 'Животное'
        verbose_name_plural = 'Животные'

    def __str__(self):
        return f"{self.unique_number} - {self.name}"

# История назначения рационов
class FeedingHistory(models.Model):
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE,
related_name='feeding_history')
    ration = models.ForeignKey(FeedingRation, on_delete=models.PROTECT)
    start_date = models.DateField()
    end_date = models.DateField(null=True, blank=True)

    class Meta:
        verbose_name = 'История кормления'
        verbose_name_plural = 'Истории кормления'

# Назначение смотрителей
class CaretakerAssignment(models.Model):
    caretaker = models.ForeignKey(Staff, on_delete=models.CASCADE,
limit_choices_to={'staff_type': 'caretaker'})
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE)
    start_date = models.DateField()
    end_date = models.DateField(null=True, blank=True)
```

```
    class Meta:
        verbose_name = 'Назначение смотрителя'
        verbose_name_plural = 'Назначения смотрителей'

# Назначение ветеринаров
class VeterinarianAssignment(models.Model):
    veterinarian = models.ForeignKey(Staff, on_delete=models.CASCADE,
limit_choices_to={'staff_type': 'vet'})
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE)
    start_date = models.DateField()
    end_date = models.DateField(null=True, blank=True)

    class Meta:
        verbose_name = 'Назначение ветеринара'
        verbose_name_plural = 'Назначения ветеринаров'

# История пересадок
class TransferHistory(models.Model):
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE,
related_name='transfer_history')
    from_enclosure = models.ForeignKey(Enclosure, on_delete=models.PROTECT,
related_name='transfers_from')
    to_enclosure = models.ForeignKey(Enclosure, on_delete=models.PROTECT,
related_name='transfers_to')
    transfer_date = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name = 'История пересадок'
        verbose_name_plural = 'Истории пересадок'
```